

FROST-GKR: A Global Trace Protocol for Batched Poseidon2b

Frobenius Reduction over Shifted Tables in $\text{GF}(2^{128})$

Andrew Boyle
dev@parano1d.org

August 2026

Abstract

We present FROST-GKR (Frobenius Reduction over Shifted Tables), a protocol for reducing a batch of width-four Poseidon2b executions over $\mathbb{F} = \text{GF}(2^{128})$ to openings of three committed multilinear polynomials. The batch is represented on one Boolean product domain indexed by permutation slot, round, and state lane. A degree-nine zero-check enforces the complete nonlinear and linear round relation. Its round-shifted terminal evaluations are then returned to the original committed columns by one degree-two sumcheck.

For a trace with $N = 2^n$ padded cells, the prover performs $O(N)$ field operations and the two constraint reductions use $2n$ sumcheck rounds, independently of the number of live permutations except through n . With full round-polynomial coefficient vectors and a generic three-column terminal batching layer, the algebraic transcript contains $22n + 18$ field elements. We prove completeness and interactive soundness. The algebraic error is at most $(18n + 14)/|\mathbb{F}|$; for the 15-variable instance evaluated in this paper, it is less than 2^{-119} before the soundness terms of the endpoint relation and polynomial commitment.

The same state commitment can be constrained at its endpoints, so the protocol composes directly with sequential chains, trees, sponge constructions, and other application graphs. A reference implementation on 59 Poseidon2b permutations reduces 472 per-permutation constraint sumchecks to two, reduces the uncompressed algebraic transcript from 287,712 to 5,568 bytes, and reduces median time for the reduction prover from 1,605.931 to 150.218 ms on an Intel Core i7-1365U.

1 Introduction

Proof systems built from hash-based state, Merkle structures, and transcripts repeatedly evaluate the same cryptographic permutation. The individual executions differ only in their inputs. Their round constants, nonlinear layer, and linear maps are identical. This regularity is easy to see at the algorithmic level, but a layer-by-layer arithmetization often discards it: each permutation receives its own columns and its own chain of sumchecks, and a high-degree S-box is decomposed into auxiliary multiplication layers.

FROST-GKR treats the repeated computation as one object. All permutation slots, all rounds, and all four lanes are coordinates of a single Boolean hypercube. The witness consists of only three columns: the round-entry state, the input to each active S-box, and its output. These columns are committed before any relation challenge is sampled. One global zero-check then proves the round equations over every live cell at once.

The remaining difficulty is adjacency. A round transition reads a state at round r and writes the state at round $r + 1$. Incrementing a binary round index is not an affine map of the multilinear variables, so an evaluation of a shifted table is not generally an evaluation of the original polynomial at a simply transformed point. FROST-GKR resolves this mismatch explicitly: the main relation may use materialized shifted views, and a second sumcheck proves

that all eleven derived evaluations are the correct linear functionals of the three original columns. The protocol therefore ends at the columns fixed by the initial commitment.

This produces a different reduction invariant from circuit-layer descent. The persistent object in FROST-GKR is the committed execution trace, and the protocol reduces global trace relations to openings of that commitment. The name records the two ingredients that make the construction effective: Frobenius squaring makes the direct x^7 relation inexpensive over a binary field, while shifted tables express the round wiring. The suffix GKR identifies the multilinear-extension and sumcheck machinery on which the reduction is built.

What the protocol proves. The internal relation certifies that every live slot is a correct Poseidon2b execution. Its state column exposes the input and output row of each slot. An application constrains those rows against the same commitment—for example, by fixing a public input, equating one slot’s output with the next slot’s input, and fixing a public digest. Acceptance then certifies the complete composed computation without repeating the permutation relation for each slot. Section 6 states this composition precisely and gives the sequential-chain equations.

1.1 Results

The main results of this paper are as follows.

1. We define a three-column committed trace relation for a batch of width-four Poseidon2b permutations. The relation covers the degree-seven S-box, round constants, full and partial linear layers, and round adjacency.
2. We reduce the whole relation with one degree-nine zero-check. For a batch padded to 2^s slots, the domain has $n = s + 7 + 2$ variables, and the main sumcheck therefore has only n rounds.
3. We give a degree-two shifted-view reduction. It batches eleven round-shifted and lane-projected evaluations and reduces them to one direct evaluation of each committed column at a new point.
4. We prove permutation semantics, completeness, topology composition, and interactive algebraic soundness. We also give exact prover, verifier, round, and transcript accounting.
5. We validate the construction in a reference implementation and compare it with a preserved per-permutation product-chain reduction on the same field, permutation sequence, and transcript channel.

2 Protocol overview

Let B be the number of live Poseidon2b executions and let $L = 2^s \geq B$ be the padded number of slots. Poseidon2b has width four and 66 nonlinear rounds. We reserve 128 round positions, so a trace cell is indexed by

$$(k, r, e) \in \{0, 1\}^s \times \{0, 1\}^7 \times \{0, 1\}^2,$$

where k is a slot, r a round index, and e a lane. The complete table has

$$n = s + 9, \quad N = 2^n = 512L$$

cells. Only $k < B$ and $r < 66$ are active constraint locations.

The witness columns are

$$s_{\text{in}}, \quad s_{\text{out}}, \quad z.$$

At an active S-box cell, s_{in} and s_{out} hold the input and output of $x \mapsto x^7$. The column z holds the state entering a round and also contains the final state row. A public selector distinguishes active from inactive S-box lanes.

The protocol has the following data flow.

Stage	Rounds	Degree	Result
Commit	—	—	Fix the three witness polynomials before relation challenges.
Global relation	n	9	Reduce every live round equation to 12 evaluations at r' .
Shift reduction	n	2	Replace eleven derived-view claims by three direct claims at r'' .
Terminal batching	$3n$	2	Produce one opening claim for each committed column.

Table 1: The FROST-GKR reduction. A polynomial commitment with a native multi-point opening can replace the last row.

The main sumcheck leaves one direct claim $Z(r')$ and eleven claims on shifted or lane-projected views. The shift sumcheck turns the latter into

$$S_{\text{in}}(r''), \quad S_{\text{out}}(r''), \quad Z(r'').$$

Thus only four point-value claims remain on the committed polynomials: two on Z and one on each S-box column. The generic terminal layer batches them into three openings.

For the evaluated batch $B = 59$, the slot axis has six variables and hence $n = 15$. The two relation sumchecks have 30 rounds in total. This should be contrasted with applying eight nine-round product sumchecks independently to each of the 59 permutations, which gives 4,248 constraint rounds before terminal batching.

3 Preliminaries

3.1 The binary field and Poseidon2b

The protocol is instantiated over

$$\mathbb{F} = \text{GF}(2^{128}).$$

Addition and subtraction coincide. The Frobenius map $x \mapsto x^2$ is linear over $\text{GF}(2)$, and

$$x^7 = x^4 x^3, \quad x^3 = x^2 x.$$

An implementation can therefore evaluate x^7 with two field multiplications and two specialized squarings.

Lemma 3.1 (Frobenius evaluation). *For all $a, b \in \mathbb{F}$,*

$$(a + b)^2 = a^2 + b^2.$$

Consequently, the direct x^7 gate requires two general multiplications when squaring is implemented as the Frobenius linear map.

Proof. In characteristic two the cross term $2ab$ is zero. Compute x^2 and x^4 by two squarings, then $x^3 = x^2 x$ and $x^7 = x^4 x^3$. $\square$

This is an implementation advantage, not a reduction in formal polynomial degree. If $a + tb$ is affine in a sumcheck variable t , then $(a + tb)^7$ has degree seven in t .

We use the width-four Poseidon2b parameter set of Grassi et al. [10]. It contains eight full rounds and 58 partial rounds, for a total of

$$R = 66.$$

Every full round applies the S-box to all four lanes; a partial round applies it only to lane zero. The initial linear map, round constants, and full and partial linear maps are fixed public parameters.

3.2 Multilinear extensions

For a table $f : \{0, 1\}^n \rightarrow \mathbb{F}$, write $\tilde{f}$ for its unique multilinear extension to $\mathbb{F}^n$. For $r, x \in \mathbb{F}^n$, define

$$\text{eq}(r, x) = \prod_{i=0}^{n-1} ((1 - r_i)(1 - x_i) + r_i x_i).$$

The extension can be evaluated as

$$\tilde{f}(r) = \sum_{x \in \{0, 1\}^n} \text{eq}(r, x) f(x). \quad (1)$$

When x is Boolean, $\text{eq}(r, x)$ is the multilinear Lagrange basis coefficient for x .

3.3 Sumcheck

For a polynomial $g : \mathbb{F}^n \rightarrow \mathbb{F}$, the sumcheck protocol of Lund, Fortnow, Karloff, and Nisan [1] reduces a claim

$$v = \sum_{x \in \{0, 1\}^n} g(x)$$

to the evaluation of g at one verifier-chosen point. If g has individual degree at most d , sumcheck uses n rounds and has interactive soundness error at most

$$\frac{nd}{|\mathbb{F}|}.$$

The prover can run in time linear in the table length when each round folds the residual tables in place.

3.4 Commitment interface

FROST-GKR is stated relative to an evaluation-binding multilinear polynomial commitment scheme. The prover commits to the three multilinear extensions

$$S_{\text{in}} = \widetilde{s_{\text{in}}}, \quad S_{\text{out}} = \widetilde{s_{\text{out}}}, \quad Z = \widetilde{z}$$

before the verifier samples any relation challenge. We denote their vector commitment by

$$C = \text{Com}(S_{\text{in}}, S_{\text{out}}, Z).$$

The algebraic protocol ends in point-value claims on these polynomials. The selected commitment scheme verifies the corresponding openings. We write ε_{pcs} for the aggregate soundness error of that interface.

In a Fiat–Shamir instantiation [12], the protocol identifier, Poseidon2b parameters, domain dimensions, complete public statement, and C are absorbed before the first challenge. Every later challenge is derived after all preceding prover messages have been absorbed.

4 The FROST trace relation

4.1 Domain and columns

Let

$$\mathcal{D} = \{0, 1\}^n, \quad n = s + 7 + 2.$$

We write a Boolean point as $x = (k, r, e)$, with slot k , round r , and lane e . The seven round bits represent $0, \dots, 127$ and the two lane bits represent $0, \dots, 3$.

For a live slot k , the witness has the following meaning.

Column	Value at (k, r, e)
z	lane e of the state entering round r
s_{in}	input of the S-box when lane e is active in round r
s_{out}	output of the S-box when lane e is active in round r

The state column contains rows $0, \dots, R$. Row zero is the state after the initial Poseidon2b linear map, and row R is the permutation output. The two S-box columns are used on rows $0, \dots, R - 1$. The remaining round positions and padded slots may be set to zero; the internal relation is masked to live cells.

The following Boolean tables are public:

- $\mu(k, r, e)$ is one exactly when $k < B$ and $r < R$;
- $\sigma(k, r, e)$ is one exactly when lane e receives the S-box in round r of a live slot;
- $\text{RC}(k, r, e)$ contains the active round constant and is zero on inactive cells;
- the coefficients of the full and partial linear maps.

Let τ increment the round coordinate modulo 128 without changing the slot or lane, and let τ^{-1} decrement it. These are permutations of the padded Boolean domain. For any table t , define the shifted and lane-projected tables

$$t^-(y) = t(\tau^{-1}(y)), \quad (2)$$

$$t_j^-(y) = t(\tau^{-1}(y) \mid \text{lane} = j), \quad j \in \{0, 1, 2, 3\}. \quad (3)$$

The second table is constant in the lane coordinate of y .

4.2 Local equations

At a source cell $x = (k, r, e)$, define

$$c_{\text{pow}}(x) = \sigma(x)(s_{\text{out}}(x) + s_{\text{in}}(x)^7) + (1 + \sigma(x))(s_{\text{out}}(x) + s_{\text{in}}(x)), \quad (4)$$

$$c_{\text{in}}(x) = \sigma(x)(s_{\text{in}}(x) + z(x) + \text{RC}(x)). \quad (5)$$

Because the field has characteristic two, $1 + \sigma$ is the complement of a Boolean selector. Thus Equation (4) imposes $s_{\text{out}} = s_{\text{in}}^7$ on an active lane and $s_{\text{out}} = s_{\text{in}}$ on an inactive lane.

For $j \in \{0, 1, 2, 3\}$, let

$$p_j(x) = \sigma(x \mid \text{lane} = j) s_{\text{out}}(x \mid \text{lane} = j) + (1 + \sigma(x \mid \text{lane} = j)) z(x \mid \text{lane} = j).$$

This chooses the S-box output on an active lane and the unchanged state on an inactive lane. For $y = \tau(x)$ with destination lane e , set

$$c_{\text{step}}(y) = z(y) + \sum_{j=0}^3 M_{\text{kind}(r)}[e, j] p_j(x), \quad x = \tau^{-1}(y). \quad (6)$$

Here $\text{kind}(r)$ selects the public full- or partial-round linear map.

Definition 4.1 (FROST internal relation). *The columns $(s_{\text{in}}, s_{\text{out}}, z)$ satisfy $\mathcal{R}_F$ if every $x \in \mathcal{D}$ with $\mu(x) = 1$ satisfies*

$$c_{\text{pow}}(x) = 0, \quad c_{\text{in}}(x) = 0, \quad c_{\text{step}}(\tau(x)) = 0.$$

Theorem 4.2 (Permutation semantics). *Fix a live slot k and a raw input $a_k \in \mathbb{F}^4$. If $\mathcal{R}_F$ holds and*

$$z(k, 0, \cdot) = M_{\text{init}} a_k,$$

then

$$z(k, R, \cdot) = \text{Poseidon2b}(a_k).$$

Conversely, every valid Poseidon2b execution extends to columns satisfying $\mathcal{R}_F$.

Proof. Consider round r . On every active lane, Equation (5) gives $s_{\text{in}} = z + \text{RC}$ and Equation (4) gives $s_{\text{out}} = s_{\text{in}}^7$. On an inactive lane, p_j selects z and is independent of the auxiliary S-box values. Equation (6) therefore applies exactly the specified full or partial linear map to the post-S-box state. Induction from row zero through row R proves the forward direction.

For the converse, record the genuine round-entry states and the S-box inputs and outputs on active lanes. Set both S-box columns to zero on inactive lanes and set padded cells to zero. Each displayed constraint then holds at every live cell. $\square$

5 The FROST protocol

The prover first sends the commitment C from Section 3.4. The protocol then performs two structural reductions: a global relation sumcheck and a shifted-view sumcheck.

5.1 Global degree-nine zero-check

After C is bound to the transcript, sample

$$\rho \leftarrow \mathbb{F}^n, \quad \beta, \gamma \leftarrow \mathbb{F}.$$

Define the Boolean table

$$u_\rho(y) = \text{eq}(\rho, \tau^{-1}(y)) \mu(\tau^{-1}(y)), \tag{7}$$

and let U_ρ be its multilinear extension.

We now express the three local constraints as polynomials in the destination coordinate Y . Every source-indexed table is replaced by the multilinear extension of its shifted view from Equation (2). Every lane lookup in the transition is replaced by the corresponding lane-projected view from Equation (3). Let

$$P(Y), \quad I(Y), \quad T(Y)$$

denote the resulting polynomial forms of, respectively, $c_{\text{pow}}(\tau^{-1}(y))$, $c_{\text{in}}(\tau^{-1}(y))$, and $c_{\text{step}}(y)$. Explicitly, the nonlinear part is

$$\begin{aligned} P(Y) &= \Sigma^-(Y)(S_{\text{out}}^-(Y) + S_{\text{in}}^-(Y)^7) \\ &\quad + (1 + \Sigma^-(Y))(S_{\text{out}}^-(Y) + S_{\text{in}}^-(Y)), \end{aligned} \tag{8}$$

and the S-box input part is

$$I(Y) = \Sigma^-(Y)(S_{\text{in}}^-(Y) + Z^-(Y) + \text{RC}^-(Y)). \tag{9}$$

For the transition, let $A_j(Y)$ be the public multilinear extension of the matrix coefficient $M_{\text{kind}(\tau^{-1}(y))}[e, j]$. Then

$$T(Y) = Z(Y) + \sum_{j=0}^3 A_j(Y)(\Sigma_j^-(Y)S_{\text{out},j}^-(Y) + (1 + \Sigma_j^-(Y))Z_j^-(Y)). \quad (10)$$

All column and schedule symbols written in uppercase denote multilinear extensions of the indicated Boolean tables; P , I , and T are the displayed composite polynomials.

The prover and verifier run sumcheck for

$$0 = \sum_{y \in \{0,1\}^n} U_\rho(y)(P(y) + \beta I(y) + \gamma T(y)). \quad (11)$$

For a valid trace, the equation follows by substituting $y = \tau(x)$ and applying Definition 4.1.

Proposition 5.1 (Degree of the main sumcheck). *The summand in Equation (11) has individual degree at most nine.*

Proof. U_ρ is multilinear. The highest-degree term is

$$U_\rho \Sigma^- (S_{\text{in}}^-)^7,$$

which has individual degree $1 + 1 + 7 = 9$. The input relation has degree at most three after multiplication by U_ρ , and the transition relation has degree at most four. $\square$

At the terminal point $r' \in \mathbb{F}^n$, the verifier checks the last sumcheck identity using the following witness evaluations:

count	evaluations
1	$Z(r')$
3	$S_{\text{in}}^-(r'), S_{\text{out}}^-(r'), Z^-(r')$
4	$S_{\text{out},j}^-(r'), j = 0, \dots, 3$
4	$Z_j^-(r'), j = 0, \dots, 3.$

The verifier computes the corresponding public-table evaluations. Only $Z(r')$ is already a direct claim on an initially committed polynomial.

5.2 Degree-two shifted-view reduction

Every derived evaluation above is a linear functional of one source table. For example,

$$S_{\text{in}}^-(r') = \sum_{x \in \{0,1\}^n} \text{eq}(r', \tau(x)) s_{\text{in}}(x). \quad (12)$$

For a lane-projected claim with lane j , the corresponding Boolean weight is

$$w_j(r', x) = \text{eq}(r'_{\text{slot}}, x_{\text{slot}}) \text{eq}(r'_{\text{round}}, \tau(x)_{\text{round}}) \mathbf{1}[x_{\text{lane}} = j]. \quad (13)$$

There is no equality factor involving r'_{lane} , because a lane-projected table is constant on that coordinate.

Enumerate the eleven claims by

i	source column	view
0	s_{in}	—
1	s_{out}	—
2	z	—
3, ..., 6	s_{out}	—, lane = 0, ..., 3
7, ..., 10	z	—, lane = 0, ..., 3.

Let c_i be the claimed value and let $w_i : \{0, 1\}^n \rightarrow \mathbb{F}$ be the corresponding public weight from Equations (12) and (13). After all c_i have been absorbed, sample $\delta \leftarrow \mathbb{F}$ and set

$$C_\delta = \sum_{i=0}^{10} \delta^i c_i.$$

For each source column $t \in \{s_{\text{in}}, s_{\text{out}}, z\}$, define

$$w_t(x) = \sum_{\{i: \text{source}(i)=t\}} \delta^i w_i(x),$$

and let W_t be its multilinear extension. The shift sumcheck proves

$$C_\delta = \sum_{x \in \{0,1\}^n} (W_{s_{\text{in}}}(x)s_{\text{in}}(x) + W_{s_{\text{out}}}(x)s_{\text{out}}(x) + W_z(x)z(x)). \quad (14)$$

Each summand is a product of two multilinear polynomials, so this is a degree-two, n -round sumcheck. At its terminal point r'' , the verifier obtains the direct claims

$$S_{\text{in}}(r''), \quad S_{\text{out}}(r''), \quad Z(r'').$$

Lemma 5.2 (Shift reduction). *If the eleven derived claims are correct, then Equation (14) holds. If at least one claim is incorrect, the random linear combination is valid with probability at most $10/|\mathbb{F}|$, apart from the soundness error of the degree-two sumcheck.*

Proof. Equation (12) follows from Equation (1) after the bijective change of variables $y = \tau(x)$. Fixing the source lane and summing the equality basis over the destination lane gives Equation (13). Multiplying the eleven identities by δ^i and summing gives Equation (14). If an identity is false, the difference is a nonzero polynomial in δ of degree at most ten. $\square$

5.3 Terminal opening claims

After the two structural reductions, the protocol has four claims:

$$Z(r'), \quad Z(r''), \quad S_{\text{in}}(r''), \quad S_{\text{out}}(r'').$$

A PCS with a native multi-point opening can discharge them directly. For a generic single-opening interface, FROST-GKR batches claims separately by source column.

For claims (p_i, v_i) on a column $T = \tilde{t}$, sample coefficients α_i after binding all claims and define

$$W(x) = \sum_i \alpha_i \text{eq}(p_i, x), \quad V = \sum_i \alpha_i v_i.$$

A degree-two sumcheck proves

$$V = \sum_{x \in \{0,1\}^n} W(x)t(x). \quad (15)$$

It terminates in one evaluation of T . Applying this once to each of Z , S_{in} , and S_{out} gives the three terminal openings used in the reference protocol.

5.4 Transcript order

The interactive message order is:

1. bind the protocol parameters and public statement;
2. commit to $(S_{\text{in}}, S_{\text{out}}, Z)$;

3. sample (ρ, β, γ) and run the main sumcheck;
4. absorb its terminal claims, sample δ , and run the shift sumcheck;
5. prove the endpoint or topology relation against the same state commitment;
6. batch the remaining claims as required by the selected PCS and verify the openings.

In particular, the commitment precedes every challenge used to combine constraints or derived evaluations.

6 Endpoint relations and composition

The state column contains the endpoint rows of every permutation, so an application can impose its topology without changing the internal Poseidon2b relation. We make this interface concrete.

Let $a_k, b_k \in \mathbb{F}^4$ be the raw input and output assigned to live slot k . The endpoint equations are

$$z(k, 0, \cdot) = M_{\text{init}} a_k, \quad z(k, R, \cdot) = b_k. \quad (16)$$

For independent hash calls, the application binds the desired a_k and b_k values. For a sequential chain, the complete linear boundary relation can be written directly on z :

$$z(0, 0, e) = \sum_{j=0}^3 M_{\text{init}}[e, j] a_{\text{pub}, j}, \quad (17)$$

$$z(k+1, 0, e) = \sum_{j=0}^3 M_{\text{init}}[e, j] z(k, R, j), \quad 0 \leq k < B-1, \quad (18)$$

$$z(B-1, R, e) = b_{\text{pub}, e} \quad \text{for every revealed output lane } e. \quad (19)$$

These are linear relations among Boolean slices of the same committed polynomial Z . They may be batched into a linear sumcheck, discharged by PCS openings, or incorporated into a larger application arithmetization. A tree, sponge, or state-transition graph replaces Equation (18) by the corresponding directed wiring equations.

Theorem 6.1 (Topology composition). *Assume that:*

1. *the committed columns satisfy $\mathcal{R}_F$ on every live slot;*
2. *a sound endpoint relation enforces Equation (16) and a directed acyclic graph among the slot endpoints; and*
3. *both relations are verified against the same evaluation-binding commitment to Z .*

Then every public sink equals the result of evaluating the specified Poseidon2b graph on its public sources.

Proof. By Theorem 4.2, every slot satisfies $b_k = \text{Poseidon2b}(a_k)$. Process the application graph in a topological order. The endpoint relation fixes each source and every subsequent input from predecessor outputs; the internal relation fixes the corresponding output. Induction determines all vertices and hence every public sink. Evaluation binding prevents the two relations from using different state columns. $\square$

Corollary 6.2 (Sequential chain). *Equations (17)–(19), together with $\mathcal{R}_F$, prove, for every revealed lane e ,*

$$b_{\text{pub}, e} = (\text{Poseidon2b}^B(a_{\text{pub}}))_e.$$

7 Completeness, soundness, and cost

7.1 Completeness

Theorem 7.1 (Completeness). *If the committed columns satisfy $\mathcal{R}_F$, all endpoint claims are correct, and the PCS opens the committed polynomials at their actual evaluations, then the honest FROST-GKR prover is accepted.*

Proof. Every local constraint vanishes on the live Boolean domain, so Equation (11) has value zero. The standard sumcheck telescope and its terminal identity therefore hold. The shifted-view identities are exact linear identities by Lemma 5.2, so Equation (14) also holds. Linearity of the equality basis gives Equation (15). The endpoint protocol and PCS accept their correct relations and openings. $\square$

7.2 Interactive soundness

Theorem 7.2 (Algebraic soundness). *Suppose the witness columns are fixed before the relation challenges, all challenges are independent and uniform in $\mathbb{F}$, and the terminal claims are discharged against an evaluation-binding PCS. For the generic three-terminal-batch protocol, the probability that columns outside $\mathcal{R}_F$ pass the FROST-GKR algebraic reductions is at most*

$$\frac{18n + 14}{|\mathbb{F}|}.$$

At $n = 15$ over $\text{GF}(2^{128})$, this is

$$\frac{284}{2^{128}} < 2^{-119}.$$

Proof. If a fixed witness violates $\mathcal{R}_F$, at least one of the three masked Boolean constraint tables is nonzero. The multilinear extension of that table vanishes at a random ρ with probability at most $n/|\mathbb{F}|$. Conditioned on a nonzero vector of the three evaluations, the combination selected by β, γ is zero with probability at most $1/|\mathbb{F}|$. The degree-nine main sumcheck contributes at most $9n/|\mathbb{F}|$.

If a shifted-view claim is false, Lemma 5.2 contributes at most $10/|\mathbb{F}|$, and the degree-two shift sumcheck contributes $2n/|\mathbb{F}|$.

For each of the three terminal column batches, a false collection of claims survives its random linear combination with probability at most $1/|\mathbb{F}|$. The three degree-two sumchecks contribute $6n/|\mathbb{F}|$. A union bound gives

$$\frac{n + 1 + 9n + 10 + 2n + 3 + 6n}{|\mathbb{F}|} = \frac{18n + 14}{|\mathbb{F}|}.$$

$\square$

Corollary 7.3 (Composed soundness). *If the endpoint relation has soundness error ε_{end} , the composed interactive argument has error at most*

$$\frac{18n + 14}{|\mathbb{F}|} + \varepsilon_{\text{end}} + \varepsilon_{\text{pcs}}.$$

Theorem 7.2 is an evaluation-soundness statement for the interactive protocol. A non-interactive deployment additionally uses the security theorem of its Fiat–Shamir transform and chosen PCS. The protocol as stated is not zero knowledge; a deployment requiring privacy must mask the committed columns and sumchecks in a manner compatible with its commitment scheme. Knowledge soundness likewise depends on the extractability guarantees of the composed commitment system.

Failure event	Upper bound
Random evaluation of a nonzero constraint table	$n/ \mathbb{F} $
Constraint combination in (β, γ)	$1/ \mathbb{F} $
Main degree-nine sumcheck	$9n/ \mathbb{F} $
Eleven-claim combination in δ	$10/ \mathbb{F} $
Shift degree-two sumcheck	$2n/ \mathbb{F} $
Three terminal claim combinations	$3/ \mathbb{F} $
Three terminal degree-two sumchecks	$6n/ \mathbb{F} $
Total	$(18n + 14)/ \mathbb{F} $

Table 2: Soundness ledger for the interactive algebraic protocol.

7.3 Asymptotic and concrete accounting

Let $N = 2^n$ be the padded number of trace cells. For fixed Poseidon2b width and round schedule, every prover stage folds a constant number of length- N tables. The total prover work is therefore $O(N)$ field operations and the committed witness contains $3N$ field elements.

The algebraic verifier processes $O(n)$ round-polynomial coefficients, plus the cost of evaluating fixed public schedules and verifying the PCS. Those public schedules can be evaluated from preprocessed fixed tables, opened from fixed commitments, or computed directly. A direct generic evaluation costs $O(N)$; the reference implementation uses this simple path.

Proposition 7.4 (Rounds and transcript elements). *The main and shift relations use $2n$ constraint sumcheck rounds. With the full-coefficient encoding and generic three-column terminal batching of Section 5.3, the protocol uses $5n$ sumcheck rounds and its algebraic transcript stores*

$$22n + 18$$

field elements before the PCS openings and serialization framing.

Proof. The degree-nine main proof stores ten coefficients in each round and 12 terminal evaluations, for $10n + 12$ elements. The shift proof stores three coefficients per round and three terminal evaluations, for $3n + 3$. Each of the three degree-two terminal batches stores three values per round and one terminal value, for $3(3n + 1)$. Their sum is $22n + 18$. $\square$

At $n = 15$, this gives 348 elements. Since one element of $\text{GF}(2^{128})$ occupies 16 bytes, the uncompressed algebraic transcript is

$$348 \cdot 16 = 5,568 \text{ bytes.}$$

A PCS that opens the four post-shift claims directly may omit the three terminal sumchecks.

8 Reference implementation and evaluation

The reference implementation serves two purposes: it validates every algebraic identity against native Poseidon2b execution, and it measures the cost of the new reduction against a preserved product-chain construction. The protocol definition and theorems above do not depend on this implementation.

8.1 Implementation

The public implementation at commit `7e66639` contains:

- native width-four Poseidon2b over $\text{GF}(2^{128})$;

- construction of the three-column unified trace;
- the main, shift, and terminal-batching provers and verifiers;
- the preserved per-permutation product-chain reduction; and
- differential tests and native checks of every terminal MLE claim.

Witness-column and transcript-facing values use a recursive tower representation. The prover converts working tables to the isomorphic polynomial basis defined by

$$X^{128} + X^7 + X^2 + X + 1.$$

The main sumcheck constructs the coefficients of $(a + tb)^7$ directly, multiplies them by the remaining affine factors, and accumulates the degree-nine round polynomial. Dedicated carry-less multiplication and Frobenius squaring are used in this flat-basis path; the tower-basis path is retained as a differential-test oracle.

The evaluation harness deliberately leaves the polynomial commitment choice open. It verifies the algebraic transcript and then evaluates the terminal MLE claims directly against the witness tables. This provides an exact correctness check for the reductions while keeping PCS-specific commitment and opening costs outside the comparison.

8.2 Baseline and method

The benchmark statement is a sequential chain of 59 Poseidon2b permutations. Both paths use the same public input, native permutation, $\text{GF}(2^{128})$ arithmetic, and Poseidon2b Fiat–Shamir channel.

The preserved baseline reduces every permutation separately. Its degree-seven S-box is represented by

$$x^2 = x \cdot x, \quad x^3 = x^2 \cdot x, \quad x^4 = x^2 \cdot x^2, \quad x^7 = x^4 \cdot x^3.$$

Five product sumchecks reduce the S-box chain and three further product sumchecks connect the resulting S-box input claims to the round-state column. Thus each permutation uses eight nine-variable sumchecks.

Before collecting timings, the executable:

1. constructs both algebraic transcripts;
2. verifies both with independent transcript instances;
3. checks that prover and verifier return identical terminal claims;
4. evaluates every terminal claim against its native witness table; and
5. asserts both exact transcript sizes.

A failure in any check aborts the benchmark.

Each reduction-verifier wrapper also evaluates the common native sequence to bind the claimed two-lane digest before checking its algebraic transcript. This step is identical for both paths.

The publication run used three warm-ups and 20 interleaved release-mode samples on an Intel Core i7-1365U with 12 logical threads, Linux 6.17.0-35-generic, Rust 1.96.0, and `-C target-cpu=native`. The CPU governor was `powersave` and turbo was enabled. Table 3 reports medians and nearest-rank p95 values. Ratios are baseline median divided by FROST-GKR median.

Operation	Product chain	FROST-GKR	Ratio
Prover	1,605.931 (1,652.096)	150.218 (162.438)	10.69×
Reduction verifier	984.269 (1,018.561)	66.499 (68.287)	14.80×
Native terminal evaluation	10.750 (11.032)	17.215 (17.539)	0.62×
Verifier and terminal evaluation	983.038 (1,018.977)	82.766 (85.361)	11.88×

Table 3: Milliseconds, reported as median (p95). The reduction verifier includes the common native digest check and ends at the terminal MLE claims; the next row evaluates those claims directly as a correctness check.

Native evaluation of the 59-permutation sequence alone has median 3.518 ms and p95 3.650 ms. It is reported separately, is outside both prover rows, and is performed inside each reduction-verifier wrapper.

Metric	Product chain	FROST-GKR	Reduction
Constraint sumchecks	472	2	236.00×
Constraint rounds	4,248	30	141.60×
All sumcheck rounds	4,263	75	56.84×
Algebraic transcript	287,712 B	5,568 B	51.67×

Table 4: Structural comparison under the common uncompressed field-element encoding. Commitment openings and serialization framing are not included in either column.

8.3 Interpretation

The structural reduction comes from two coordinated choices. The direct x^7 relation removes the product-chain columns and their dependent sumchecks, while the slot-round-lane domain lets one zero-check cover the entire batch. The benchmark measures these choices together.

The FROST-GKR terminal evaluation is slower than the baseline terminal evaluation because it reconstructs and evaluates three unified columns. This cost is a property of the native validation harness, not of a particular PCS. In the complete measured path it is outweighed by the smaller relation verifier.

The timing values are specific to the reported CPU, compiler, and 59-permutation workload. The structural counts in Table 4, by contrast, follow directly from the two proof objects. They establish that the global reduction replaces a number of per-permutation constraint sumchecks linear in B by two sumchecks whose depth grows logarithmically with the padded batch.

9 Related work

The sumcheck protocol was introduced by Lund, Fortnow, Karloff, and Nisan [1]. Goldwasser, Kalai, and Rothblum used multilinear extensions and sumcheck to verify layered circuit evaluation [2], and Thaler developed practical time-optimal refinements [3]. In those protocols, verification maintains an output claim and reduces it through circuit layers. FROST-GKR instead commits to global execution columns and reduces a trace zero-check plus the linear identities induced by shifted views.

Committed multilinear arithmetizations provide the general setting for this design. Spartan combines sumcheck with polynomial commitments for R1CS instances [4]. HyperPlonk gives Boolean-hypercube zero-checks for high-degree custom gates and reduces them to multilinear polynomial openings [5]. FROST-GKR specializes the committed zero-check approach to the Poseidon2b round schedule and adds a dedicated reduction for round-shifted and lane-projected trace views.

Belling, Soleimanian, and Bégassat study custom GKR gates for hash verification [6], while Soukhanov develops power-oriented GKR arithmetizations [7]. These works show the value of matching a sumcheck relation to a high-degree gate. FROST-GKR combines a direct degree-seven gate with batch-wide trace aggregation and explicit round adjacency.

Diamond and Posen develop succinct arguments over towers of binary fields and a general virtual-polynomial treatment of circular shifts of Boolean tables [8]. The FROST-GKR shift reduction is narrower: it batches the exact eleven derived evaluations produced by the Poseidon2b transition and returns them to three source columns with one degree-two sumcheck.

Poseidon2b was designed for binary extension fields [10], following the prime-field Poseidon2 construction [9]. Binary-tower proof systems such as Binius [8] provide a natural environment for its use. The final multilinear claims of FROST-GKR can be opened by a suitable field-compatible PCS; BaseFold is one field-agnostic example [11].

10 Conclusion

FROST-GKR is a global committed-trace protocol for repeated Poseidon2b execution over $\text{GF}(2^{128})$. Three witness columns describe an arbitrary batch, one degree-nine zero-check proves every live round relation, and one degree-two shift reduction returns all derived views to the original commitment.

The construction gives an integrating proof system a precise interface: FROST-GKR certifies permutation semantics, while linear endpoint relations connect those semantics to the application’s public inputs, outputs, and graph. Both parts use the same state commitment, so a chain, tree, sponge, or state transition can reuse one batch-wide permutation proof.

For the 59-permutation instance, the reference implementation reduces the constraint protocol from 472 sumchecks to two and the uncompressed algebraic transcript by a factor of 51.67, with a 10.69-fold median prover speedup for the reduction prover against the preserved product-chain construction. The result is a compact, linear-time algebraic backend for proof systems whose computation contains many Poseidon2b evaluations.

Acknowledgements

This work originated in the proof system developed for the Parano1d project. The illustrated protocol overview, technical documentation, and reference implementation and benchmark report are available online.

References

- [1] C. Lund, L. Fortnow, H. Karloff, and N. Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM*, 39(4):859–868, 1992. <https://doi.org/10.1145/146585.146605>.
- [2] S. Goldwasser, Y. T. Kalai, and G. N. Rothblum. Delegating computation: Interactive proofs for muggles. *Journal of the ACM*, 62(4), Article 27, 2015. Preliminary version in STOC 2008.
- [3] J. Thaler. Time-optimal interactive proofs for circuit evaluation. In *Advances in Cryptology—CRYPTO 2013*, pages 71–89, 2013.
- [4] S. Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Advances in Cryptology—CRYPTO 2020*, 2020. <https://eprint.iacr.org/2019/550>.

- [5] B. Chen, B. Bünz, D. Boneh, and Z. Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In *Advances in Cryptology—EUROCRYPT 2023*, pages 499–530, 2023. <https://eprint.iacr.org/2022/1355>.
- [6] A. Belling, A. Soleimanian, and O. Bégassat. Recursion over public-coin interactive proof systems; faster hash verification. In *ACM CCS 2023*, pages 1422–1436, 2023. <https://eprint.iacr.org/2022/1072>.
- [7] L. Soukhanov. Power circuits: A new arithmetization for GKR-styled sumcheck. Cryptology ePrint Archive, Report 2023/1611, 2023. <https://eprint.iacr.org/2023/1611>.
- [8] B. E. Diamond and J. Posen. Succinct arguments over towers of binary fields. In *Advances in Cryptology—EUROCRYPT 2025, Part IV*, pages 93–122, 2025. <https://eprint.iacr.org/2023/1784>.
- [9] L. Grassi, D. Khovratovich, and M. Schofnegger. Poseidon2: A faster version of the Poseidon hash function. Cryptology ePrint Archive, Report 2023/323, 2023. <https://eprint.iacr.org/2023/323>.
- [10] L. Grassi, D. Khovratovich, K. Koschatko, C. Rechberger, M. Schofnegger, V. Schröppel, and Z. Wu. Poseidon(2)b: Binary field versions of Poseidon/Poseidon2. *IACR Communications in Cryptology*, 2(4), 2026. <https://doi.org/10.62056/a66ce0zn4>.
- [11] H. Zeilberger, B. Chen, and B. Fisch. BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In *Advances in Cryptology—CRYPTO 2024, Part X*, pages 138–169, 2024. <https://eprint.iacr.org/2023/1705>.
- [12] A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology—CRYPTO 1986*, pages 186–194, 1987.